Verdandi

norns on the Mac: a community port, as a standalone app and an AUv3 plugin, with the script catalog. not affiliated with monome. free software under the GPL-3.0.

1. what this is
2. installing
3. first start
4. norns in five minutes
5. controls
6. audio
7. maiden and the script catalog
8. the script builder
9. scripts
10. MIDI, grid, arc, crow, gamepads
11. the AUv3 plugin
12. files and logs
13. troubleshooting
14. differences from the device
15. uninstalling
16. source and licenses

appendix: script builder reference

---

1. what this is

norns is a sound computer by monome: a small box running Lua scripts, a SuperCollider audio engine and the softcut sample looper, with a 128×64 screen, three keys and three encoders. Verdandi runs the same software natively on macOS. the window shows the device: the screen behind its bezel, the keys and encoders where they sit on a norns, played with the mouse or the computer keyboard. the community scripts run unchanged.

the name: norns is monome's, and this port is not theirs, so it has a name of its own. Verdandi is one of the three norns of the old stories, the one of the present, of what is happening now.



everything that makes a norns a norns is here: matron (scripts, screen, devices), crone and softcut (audio routing, loopers, tape), SuperCollider with the sc3-plugins, and maiden (the web editor with the script catalog). in addition Verdandi installs an AUv3 plugin that puts norns into a DAW track.

new to norns? norns in five minutes has the basics; monome's norns documentation has the rest, and it applies to Verdandi too.

---

2. installing

requirements: a Mac with Apple silicon (M1 or later), macOS 15 Sequoia or newer.

1. open `Verdandi.dmg` and drag **Verdandi** onto **Applications**.
2. the app is not notarized by Apple (it is ad-hoc signed, without a developer identity), so macOS blocks the first start. open Verdandi once and close the message, then open *System Settings* → *Privacy & Security*, scroll down to Verdandi and click **Open Anyway**, and confirm. (macOS 15 no longer offers right-click → Open for this.) if macOS says the app is damaged, run `xattr -dr com.apple.quarantine /Applications/Verdandi.app` in Terminal.
3. when Verdandi asks for the microphone, allow it: that is the audio input for softcut, tape and engines. (you can change it later under *Privacy & Security* → *Microphone*.)

updates: when Verdandi starts (and once a day while it runs) it asks GitHub whether there is a newer release, and if so says it on the window: **Download** opens the release page, **Later** asks again at the next start, **Skip This Version** waits for the one after. it sends nothing but the app's version. to update, drag the new Verdandi onto Applications and replace the old one; dust in `~/norns` stays as it is. *Verdandi* → *Check for Updates Automatically* turns the check off.

3. first start

Verdandi starts the audio server, the SuperCollider engine and maiden, then opens its window with the norns device. the first start takes a little longer: macOS checks the bundled libraries once.

your norns lives in the folder `~/norns` in your home folder, laid out like `/home/we` on the device:

path	contents
`~/norns/dust/code`	installed scripts
`~/norns/dust/audio`	samples, tape recordings (`audio/tape`)
`~/norns/dust/data`	script data, PSETs, system settings
`~/norns/norns`	a link into the app (the norns core); do not edit
`~/norns/*.log`	logs, see files and logs

to stop Verdandi, close its window, quit it from the Dock, or choose SLEEP in the norns menu. only one Verdandi runs at a time.

4. norns in five minutes

Verdandi plays like a norns. what monome's norns documentation says holds here too, apart from the few things in differences from the device. the short version:

- **a script plays, the menu waits.** norns runs one script at a time, and the screen, keys and encoders belong to it. a short press on K1 (`1`) switches between the script and the menu.
- **in the menu** E1 goes through the pages MIX, TAPE, HOME and PARAMS (the buttons above the device do the same), E2 scrolls, K3 enters or confirms, K2 goes back.
- **HOME:** SELECT picks a script (K3 loads it), SYSTEM has devices, audio, mods, reset and restart, SLEEP quits Verdandi.
- **PARAMS:** everything the script lets you set, then norns' own settings (levels, reverb, compressor, clock). PSET saves and loads them as presets.
- **MIX:** the levels of out, in, monitor, engine, softcut and tape.
- **TAPE:** records what norns plays into `dust/audio/tape` , and plays files back.
- **new scripts** come from the catalog in maiden; then SELECT them. how a script is played is at the top of its file, in maiden's catalog entry, or on its page on lines.
- **K1 + turn:** many scripts do something else while K1 is held: hold `1` and turn.

more, from monome: playing norns (the menus in depth), maiden, clocks and controllers,

writing scripts, the API reference and help. questions about Verdandi itself go to its GitHub issues, not to monome.

---

5. controls

keyboard

while the Verdandi window is in front, the keyboard is the norns hardware:

keys	norns
`1` `2` `3`	K1 K2 K3 (held while you hold the key, so K1 + turn works)
`↩` (Return) or `Space`	K3: confirm
`Esc` or `⌫` (Backspace)	K2: back
`Q` / `W`	E1 left / right
`A` / `S`	E2 left / right (in scripts also `↑` / `↓`)
`Z` / `X`	E3 left / right (in scripts also `←` / `→`)
with `⇧`	5 steps at once; holding a key keeps turning
`F1`–`F4`, `F5`	menu pages MIX, TAPE, HOME, PARAMS; F5 toggles the menu
`F6` `F7` `F8`	open or close the virtual grid, keyboard and arc

this works wherever nothing else wants the keys. when a text entry is open (naming a file, searching), you type. scripts that use the keyboard themselves (live coding, keyboard instruments, about 14 of the 350 catalog scripts) get the keys; there, hold `⌥` (Option) with the keys above, or press `⌥` `Esc` to force hardware keys on or off (the window title then reads *Verdandi · keys*).

mouse

the keys and knobs in the window are the controls:

- **keys:** click K1, K2 or K3. the key stays down while you hold the mouse button.
- **knobs:** drag a knob up or right to turn it clockwise, down or left for counter-clockwise. or scroll over it. scrolling anywhere else turns the nearest knob. hold `⇧` while dragging for fine steps, `⌘` for fast ones.
- for K1 + turn, hold `1` on the keyboard and turn a knob with the mouse.
- **the MIX page:** drag a level bar on the screen up or down (out, in, mon, eng, cut, tp), or scroll over it; a double click sets it back to 0 dB.

- **OUT and MUTE** (right of E3, not on the real norns): the fader is norns' output level, the same as *out* on the MIX page: drag it or scroll over it, double click it for 0 dB. MUTE silences the output and brings it back at the level it had; turning the level up also ends the mute. norns starts with the level from before the mute.

the keys and knobs in the window always show what norns receives, also from the keyboard.

the buttons above the device

the real norns does not have these; they save trips through the menu:

button	does
MIX TAPE HOME PARAMS	opens that menu page (its light shows the page you are on). click the lit one to go back to the script.
MIC	chooses what norns' input hears; each click goes on to the next: **MIC** (the Mac's microphone or your interface's inputs), **SYS** (the sound of every other app: a video in the browser, a synth, Music) and **OFF**. **APP** means one app chosen in Devices → Audio Input, see other apps as input. its light is on while something reaches norns. switch the microphone off when speakers are loud and close to it, so they cannot feed back. norns starts with what it had.
GRID ARC KEYS	open or close the virtual grid, arc and keyboard (also `F6`, `F8`, `F7`), see devices. the light is on while one is open. norns and the open devices are laid out together so all fit the screen; when the last one closes, norns goes back to where it was.
MAIDEN	opens maiden in your browser.
MANUAL	opens this manual.

the menu bar

menu	does
Verdandi → Check for Updates…	asks GitHub for a newer Verdandi now (see updates); *Check for Updates Automatically* below it turns the check at start and once a day on or off.
File → Import Audio or Script… ⌘O	adds audio and scripts to norns, like dropping them on the window (see files); tick *Copy audio into norns* for a copy instead of a link.
File → Show dust Folder ⇧⌘D	opens norns' dust folder (scripts, audio, data) in the Finder.
File → Show Tape Recordings ⇧⌘T	opens the folder TAPE records into.
File → Save Screenshot ⇧⌘S	saves the screen as a sharp 512×256 picture on your desktop, for sharing a script on lines or norns.community.
File → Record GIF ⇧⌘G	records the screen as an animated GIF (up to a minute, a red dot shows it is recording); the same item stops it and saves it on your desktop.
Devices	opens or closes the grid, keyboard and arc.
Devices → Audio Input	what norns' input hears, as with the MIC button, or one app of those running; see other apps as input.
View → Device Skin / Dark Flat Skin	how the windows look: as the aluminium device, or dark and flat (easier on the eyes in a dark room). the grid, arc and keyboard follow; the choice is kept for the next start.

window

the window can be resized and keeps the shape of the device; the screen is always drawn with sharp pixels. **View → Dark Flat Skin** draws it dark and flat instead of aluminium.

6. audio

norns starts with the Mac's **default input and output** (*System Settings → Sound*) at 48 kHz with a 256-frame buffer (about 5 ms). to use an audio interface, another sample rate or a smaller buffer, go to **SYSTEM → AUDIO**:

- **E2** picks a line, **E3** changes it: INPUT, OUTPUT (an interface or *default*), RATE (44.1 to 96 kHz), BUFFER (64 to 1024 frames, with the latency next to it).
- go to APPLY and press **K3**: norns restarts its audio with the new setting, which is kept for the next start.
- if the setting does not work (say the interface cannot do that rate), norns goes back to the defaults and the AUDIO page says so. an interface that is unplugged is replaced by the Mac's default device.

a mono microphone (like the built-in one) feeds both norns inputs. levels, monitoring, reverb and compressor are in the norns MIX menu, as on the device.

other apps as input

to loop, sample or process what other apps play (a video in the browser, a synth, a drum machine, Music) without a DAW or a loopback driver, click MIC until it says **SYS**: norns' input hears the sound of every app but norns itself. for one app only, choose it in **Devices → Audio Input** (apps that are playing right now are at the top); the button says **APP**.

- the sound goes into norns' inputs instead of the microphone or interface. scripts record and process it like anything plugged into a norns: record a buffer in a looper while a video plays, and it is in the buffer.
- the apps keep playing as before: you hear them directly, and norns gets a copy. norns' input monitor of them goes to TAPE only, so nothing is heard twice: TAPE records the video together with what norns plays, softcut and engines get it as their input.
- the app does not have to play, or even run, yet: norns picks it up when it starts.
- the first time, macOS asks whether Verdandi may record audio from other apps; allow it. if the app stays silent in norns, check *System Settings → Privacy & Security → Screen & System Audio Recording*.
- while a DAW plays through the AUv3 plugin, norns' input is the DAW track: SYS and APP pause meanwhile, and every app (the DAW too) is heard directly.

one app at a time. SYS and APP are kept for the next start; a note on the device says so.

---

7. maiden and the script catalog

maiden is norns' web editor. with Verdandi running, open `http://127.0.0.1:5000/maiden` in your browser.

- **project manager** (the box icon): browse the community catalog, install and update scripts. installed scripts appear in the norns SELECT menu.
- **editor**: edit everything in `dust`.
- **REPL**: the matron (Lua) and SuperCollider consoles; errors from scripts show up here.

unlike on the device, maiden and the consoles only answer on this Mac (127.0.0.1), not on your network: the consoles can run any code.

the **blocks** button at the bottom right of maiden opens the script builder: see 8. the script builder.

---

8. the script builder

norns scripts without writing Lua, in maiden: the orange **blocks** button at the bottom right of maiden, or `http://127.0.0.1:5000/maiden/blocks/` (its **manual** link opens this chapter). every block, module and screen element is listed in the appendix.

8.1 what it is

The script builder makes ordinary norns scripts from three pages. You switch between them at the top:

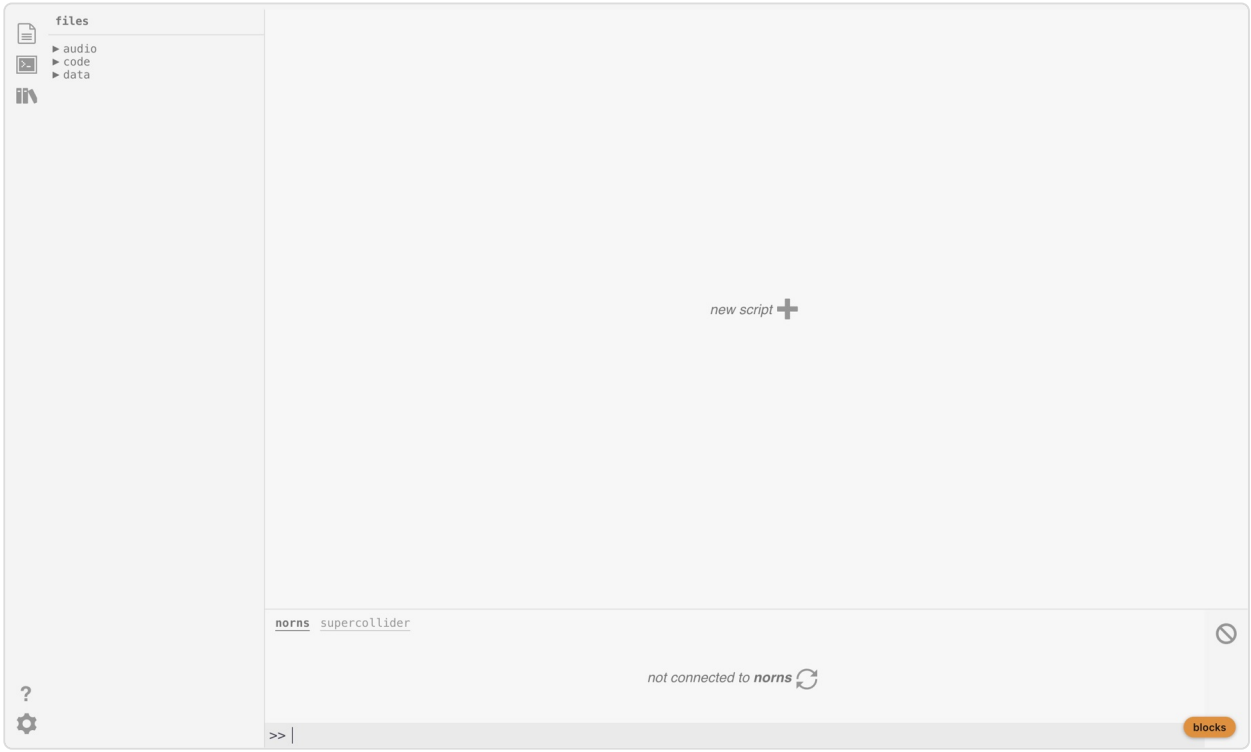
- **blocks**: *what the script does*. Blocks that snap together: when the script starts, every quarter beat, when a key is pressed, when an encoder turns, when a MIDI note arrives, and what happens then: play a note, change a sound, record a loop, light a grid key, draw something.
- **sound**: *how it sounds*. An instrument of modules joined by cables (oscillators, filters, envelopes, LFOs, a sampler, effects), played by Weave, the engine that comes with the builder.
- **screen**: *how it looks*. Elements placed on the 128 × 64 norns screen, and things that move: rings on every note, stars, orbits, waves, meters. Any number can move with the beat, on each note, with the sound.

What you build is a real norns script: `code/<name>/<name>.lua` in maiden, with your blocks next to it (`<name>.blocks.json`), so you can open it in the builder again. The **lua** tab on the right always shows the script your blocks make, line by line.

The builder runs on your Mac only: maiden listens on `127.0.0.1` , nobody else can see your scripts until you share them.

8.2 opening it

1. Start Verdandi.
2. Open maiden: the **MAIDEN** button above the norns screen, or `http://127.0.0.1:5000/maiden` in a browser.
3. Click the orange **blocks** button at the bottom right. The builder opens at `http://127.0.0.1:5000/maiden/blocks/` (bookmark it), with the example *first_steps*.

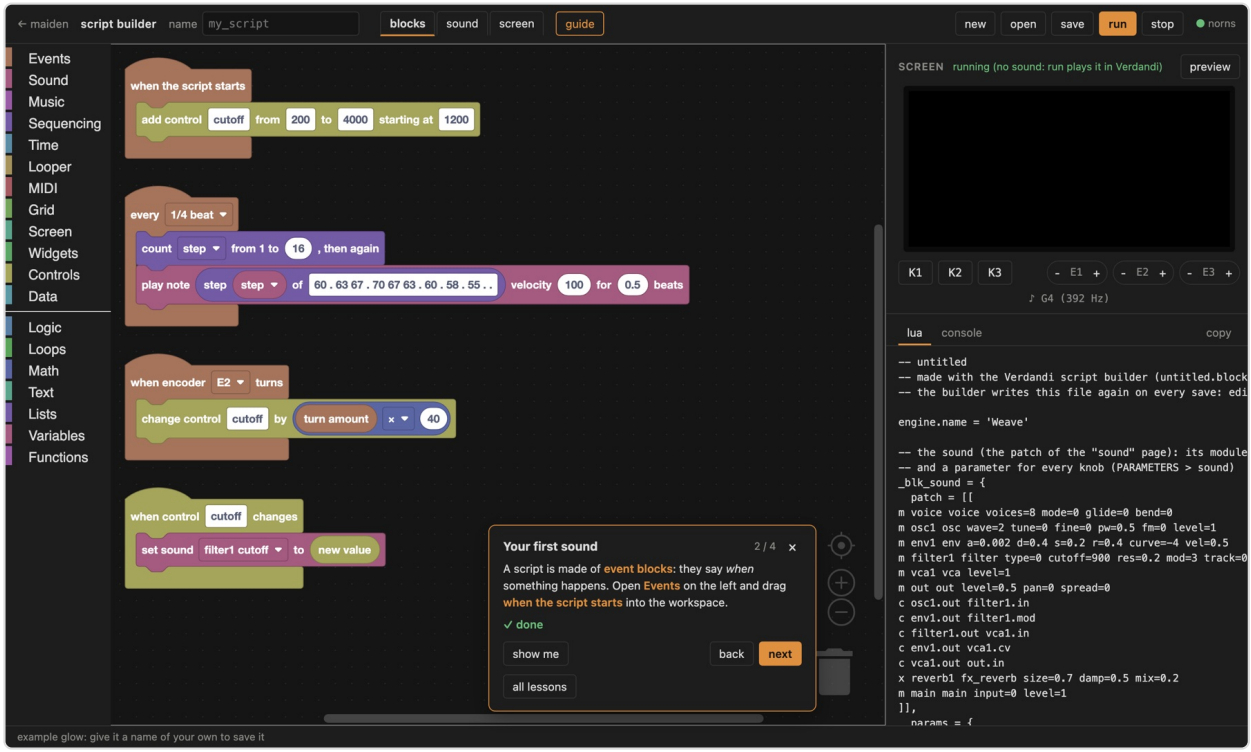


maiden: the orange **blocks** button opens the builder.

The page has the blocks (or the sound, or the screen) on the left and, on the right, a **preview** of the norns screen with K1 to K3 and E1 to E3 you can click (scroll over an encoder to turn it), the **lua** the blocks make and the **console** of norns. The preview runs your script in the page, silently: you see notes as `♪ C3` under the screen. **run** plays it in Verdandi, with sound.

8.3 the guide

guide at the top opens lessons in small steps: your first sound, a melody, the sound page, an encoder, drawing, the screen page, a looper, a grid, saving. Each step says where its blocks are (**show me** opens the right group of the toolbox) and notices by itself when you have done it.

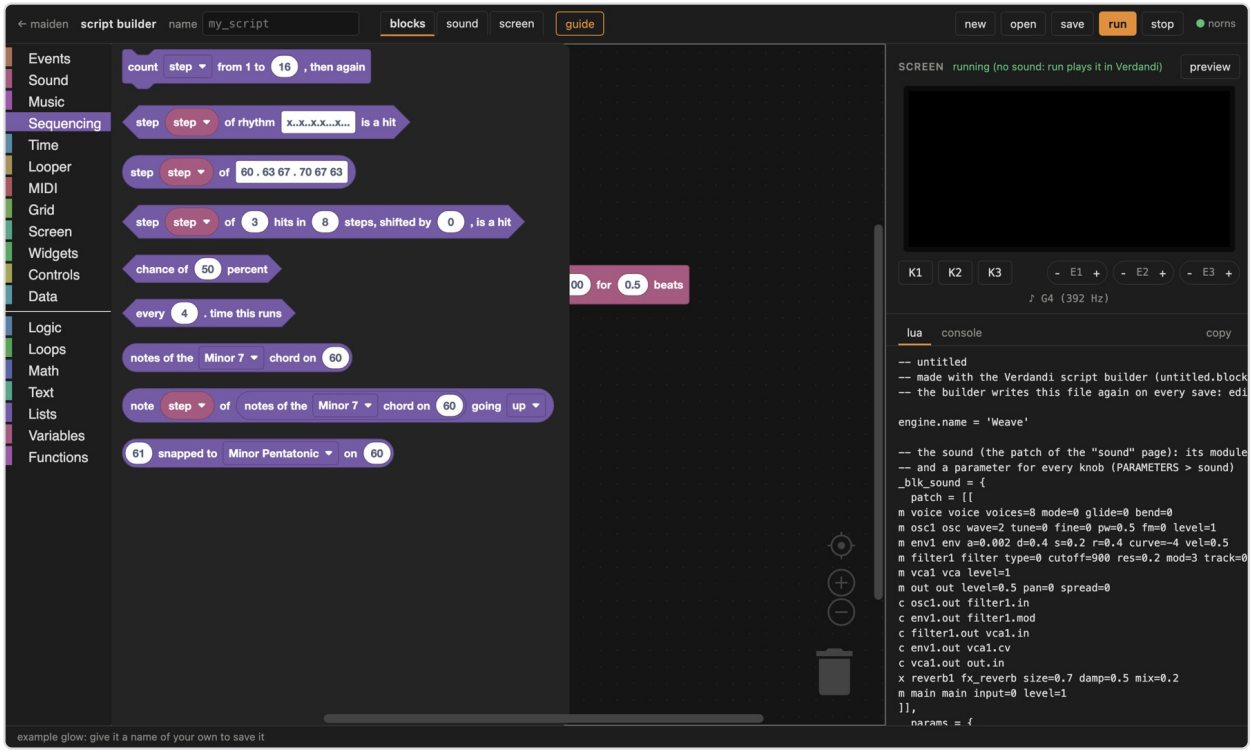


the guide in the corner of the blocks; it ticks a step off when it is done.

8.4 tutorial 1: the blocks

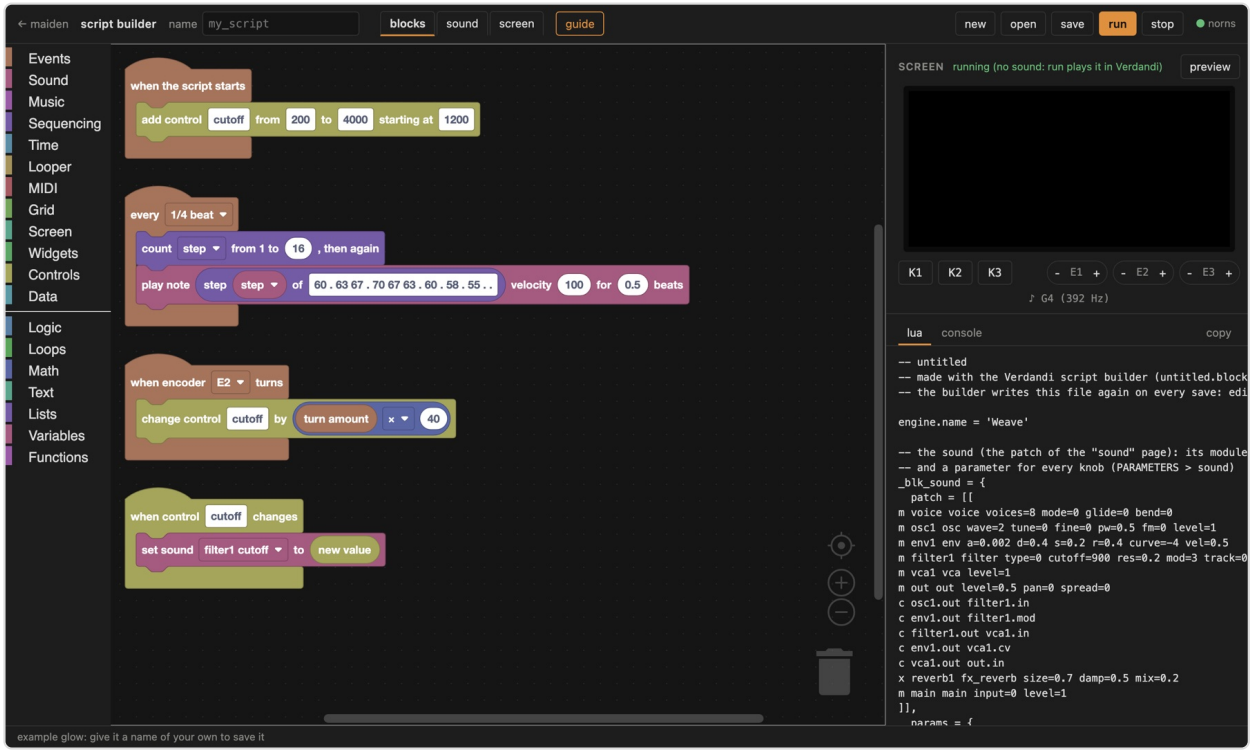
This tutorial builds *glow*: a melody, a sound with a moving filter and an echo, a screen that glows with every note, and the encoder E2 on the filter. **open > glow** loads the finished one. To build it yourself, press **new**.

The toolbox on the left holds the blocks by group. Click a group, drag a block into the workspace. A block with a notch at the top snaps under another one; a round slot takes a value (a number, a note, a block that gives a value). Event blocks, with a rounded top, start a stack: everything inside them runs when their event happens.



the toolbox with **Sequencing** open: patterns, euclidean rhythms, chords, arpeggios, chance.

1. **A control.** From **Events**, drag **when the script starts**. From **Controls**, put **add control cutoff from 200 to 4000 starting at 1200** into it. A control is a number with a range; it shows in PARAMETERS on the norns, MIDI can move it, and a PSET saves it.
2. **A beat.** From **Events**, drag **every 1/4 beat**. What is inside repeats in time with the norns clock (the tempo is in PARAMETERS > CLOCK).
3. **A counter.** From **Sequencing**, put **count step from 1 to 16, then again** inside it. Each quarter beat, `step` goes up by one, back to 1 after 16.
4. **A melody.** From **Sound**, put **play note … velocity 100 for 0.5 beats** under the counter. Into its note slot drop **step … of 60 . 63 67 . 70 67 63 . 60 . 58 . 55 . .** (Sequencing) and into that one's first slot the variable **step** (Variables). The text is the melody: note numbers with spaces between them, 60 is middle C (C3 on norns), a dot is a rest.
5. **An encoder.** From **Events**, drag **when encoder E2 turns**; put **change control cutoff by turn amount × 40** into it (Controls, Math).
6. **The control moves the sound.** From **Controls**, drag **when control cutoff changes**, and put **set sound filter1 cutoff to new value** (Sound) into it.



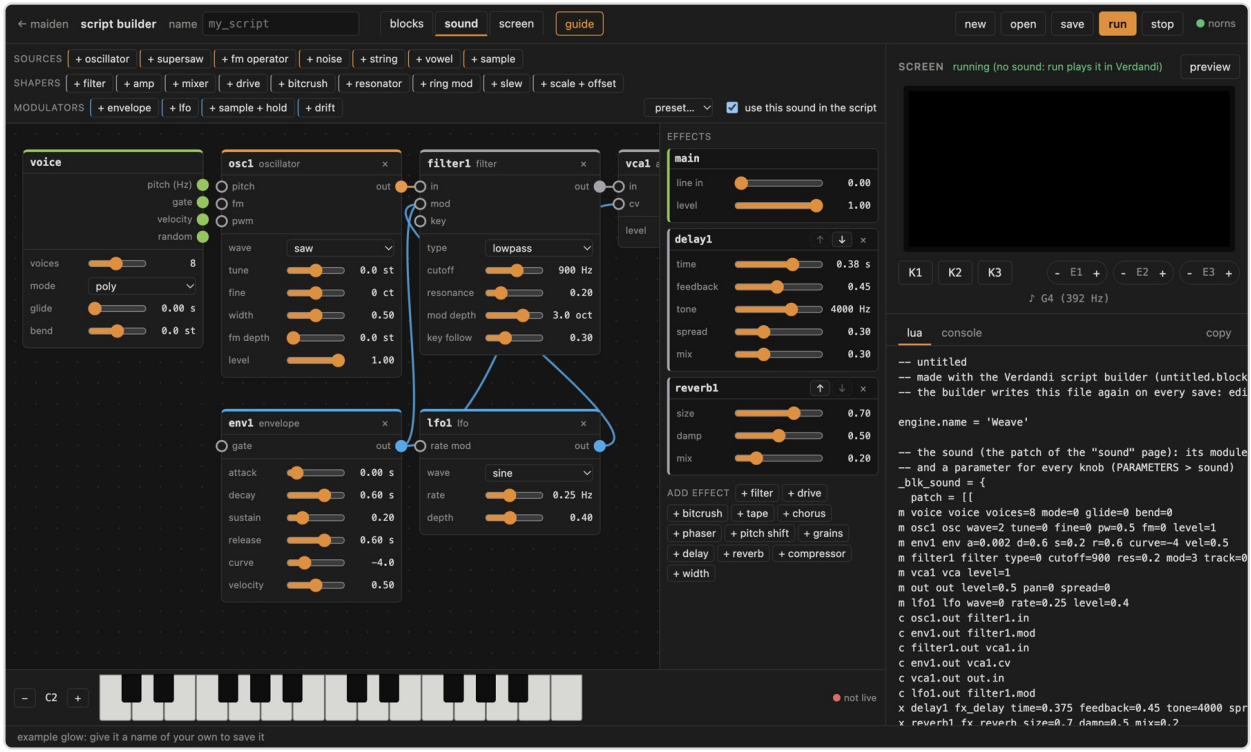
the blocks of glow. The preview on the right plays it silently; the notes show under the screen.

> A block that is in the wrong place gets a yellow warning (for example **turn amount** outside **when encoder … turns**). A block outside every event block is greyed out: it never runs. Hover over a block for what it does.

8.5 tutorial 2: the sound

Click **sound** at the top. This page is the instrument your notes play. Every note gets a **voice** of its own: the **voice** module gives each one its pitch, its gate (held or not) and its velocity; whatever reaches **output** is heard, through the effects on the right. Up to 16 voices play at once; **mode** mono or legato plays one at a time, with **glide**.

1. A new script starts with **pluck**: a saw (oscillator) through a filter and an amp, both moved by an envelope. The **preset…** menu has more: pad, bass, bell, string, drone, kick, choir, sampler, acid, bleeps.
2. **Add a module.** Click **+ lfo** (modulators). An LFO is a slow wave.
3. **Plug a cable.** Drag from the round jack **out** on the right of the LFO to **mod** on the left of the filter. The filter now opens and closes by itself. Click a cable to remove it. Cables go from an output to an input; an input can take several (they add up). A cable that would make a loop is refused.
4. **Turn knobs.** LFO **rate** 0.25 Hz, **depth** 0.4. Double-click a slider: back to its default.
5. **An effect.** Under **add effect**, click **+ delay**; the arrows move an effect up or down the chain.



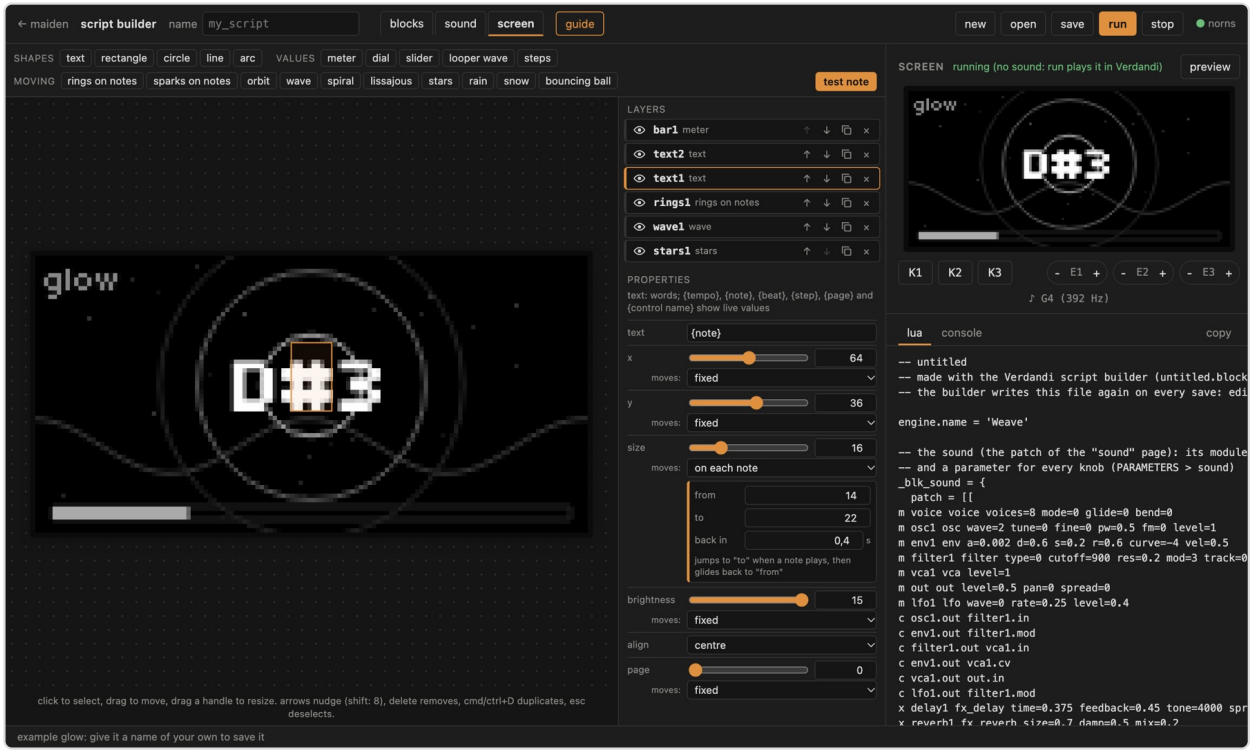
the sound page of glow: the voice, oscillator, filter, envelope, amp and the LFO on the filter; delay and reverb on the right; the keyboard at the bottom.

Every knob is also a control: PARAMETERS > sound on the norns, mappable to MIDI, and in the blocks **set sound … to**, **change sound … by** and **sound …** reach it. While the script runs in Verdandi, the knobs and the keyboard at the bottom play **live**; changing a cable starts the script again by itself.

8.6 tutorial 3: the screen

Click **screen** at the top. The big screen shows exactly what the norns will show, live. Add elements with the buttons at the top; click one to select it, drag it to move it, drag its corners to resize it, arrow keys move it a pixel (shift: 8). The layer list on the right puts one over another; its eye hides one.

1. **stars** (moving): a starfield behind everything.
2. **rings on notes**: every note sends a ring out. **test note** shows one without playing.
3. **text** with `{note}`: the name of the note just played. In the properties, set **size** to move **on each note**, from 14 to 22: the name jumps with every note and shrinks back.
4. **wave**: set its **height** to move **with the sound level**.
5. **meter**: its **value** moves **with a control**: `cutoff`. Turn E2: the bar follows.



the screen page of glow, the note name selected: its size moves on each note, from 14 to 22, back in 0.4 s.

Any number of an element can be **fixed** or move: **with the beat** (smooth, triangle, saw, square, random steps; every so many beats), **over time** (the same in seconds), **on each note**, **with the sound level**, **with a control**, or **with a looper's playhead**. Text shows live values in braces: `{note}`, `{tempo}`, `{beat}`, `{page}`, `{step}` (any variable of the blocks) and `{cutoff}` (any control). Blocks under **Screen** change pages, hide and show elements and set their values; draw on top of the design with **draw the screen**.

8.7 tutorial 4: run it in Verdandi

1. Type a **name** at the top: `glow` (letters, numbers, `_`).
2. Press **run**. The builder saves the script and starts it in Verdandi: you hear it, and the norns screen shows it. The **console** shows what norns says (errors in red).
3. Turn E2 in Verdandi (or on a MIDI controller mapped to `cutoff`): the filter and the meter follow.
4. On the sound page, the knobs and keyboard now play live. **stop** ends the script.



glow running in Verdandi: the screen design on the norns screen, E2 on the filter and the meter.

> The dot next to **norns** at the top right is green when the builder reaches Verdandi. Red: start Verdandi (or restart it), the builder connects again by itself.

8.8 saving, opening, files

- **save** (⌘S) writes `code/<name>/<name>.lua` and `<name>.blocks.json` through maiden. **run** (⌘↩) saves first. maiden shows them under `code`; on the norns they are in SELECT like any script.
- The builder never writes into a folder it did not make: a name taken by another script is refused.
- **open** lists the scripts made here and the examples (first_steps, pulse, looper, drums, grid_keys, glow, visuals).
- Your blocks are also kept in the browser as you work: closing the page loses nothing; it opens with them again.
- The `.lua` is written again on every save: change the blocks, not the text. To go on in Lua, copy it to a folder of its own and edit it in maiden.

8.9 taking a script to a norns

A script from the builder runs on any norns. Copy its folder `code/<name>` to the norns like any script (maiden on the norns, or SMB/SFTP to `dust/code`). A script with a sound needs the Weave engine on that norns, once: copy `Engine_Weave.sc` and

`WeaveGraph.sc` (in Verdandi: `norns/sc/engines/`) to `dust/code/weave/lib/` on the norns, then SYSTEM > RESTART. Verdandi has Weave built in.

8.10 tips and trouble

what	why, and what helps
no sound in the preview	the preview is silent on purpose; **run** plays it in Verdandi.
"norns is not reachable"	Verdandi is not running, or maiden is open as `localhost`: use `127.0.0.1`.
"ran without pausing for too long"	a loop never waits: put **wait** in it, or use **every ...**.
a block is greyed out	it is outside every event block: it never runs.
a yellow warning on a block	it is somewhere it cannot work (hover over the warning).
"sound ... is not on the sound page any more"	its module was removed: pick another knob in the block.
"no control ..."	add the control in **when the script starts** before using it.
a looper records nothing	it records the audio input: check SYSTEM > AUDIO and the microphone permission.

9. scripts

scripts run unchanged. the port takes care of what differs from the device:

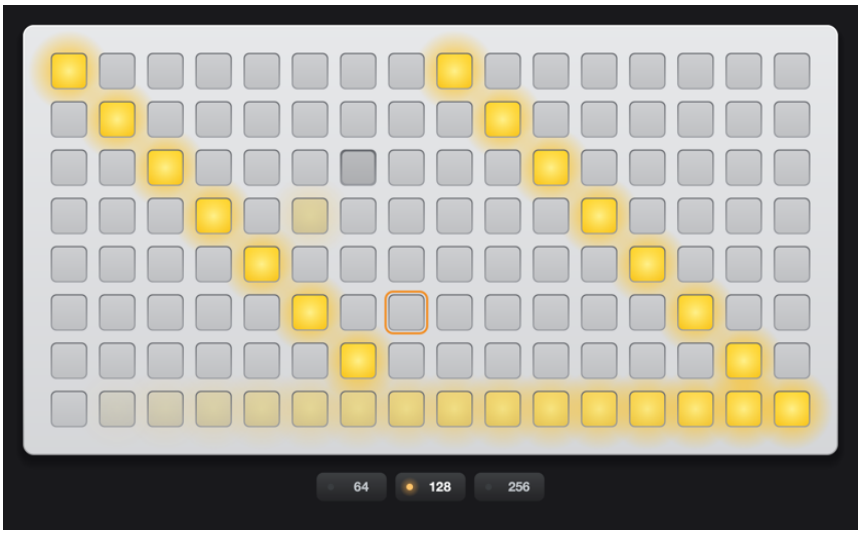
- scripts see the usual `/home/we/dust/...` paths; norns maps them to `~/norns`.
- the sc3-plugins (Decimator, MoogLadder, Greyhole, ...) are built in.
- `require "cjson"` uses the built-in JSON library.
- shell commands find the same GNU tools as on the device.

scripts that bring compiled Linux programs or Linux-only UGens, or that install software with `apt`, can fail in those parts; everything else in them still works. an engine that fails to load blocks later engine loads until you restart norns (as on the device).

new engines are compiled when norns starts: after installing a script with its own engine, restart norns (or use SYSTEM → RESTART in the menu).

10. MIDI, grid, arc, crow, gamepads

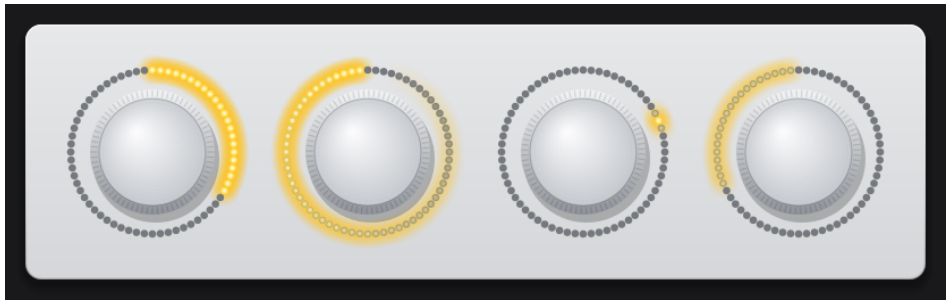
- **MIDI**: class-compliant USB MIDI devices and network/IAC ports appear in SYSTEM → DEVICES → MIDI. norns also creates a virtual port **norns** that other apps (a DAW) can send to and receive from.
- **grid and arc**: plug them in; no serialosc needed.
- **no grid?** the GRID button (or `F6`) opens a virtual grid in its own window, as a 64 (8×8), 128 (16×8) or 256 (16×16): pick the size with the switch under it (norns keeps the choice; a script that is running may need to be loaded again to use a new size). norns and the scripts see it as a real grid (in SYSTEM → DEVICES it is *virtual grid*). click a key to press it, drag across keys to play them in turn, right-click (or ctrl-click) to hold a key down, for chords; right-click again to let it go. closing the window unplugs it.



- **no MIDI keyboard?** the KEYS button (or `F7`) opens a two octave keyboard. norns and the scripts see it as the MIDI device *Verdandi keys* (other apps, a DAW, see it too); it takes the first free port in SYSTEM → DEVICES → MIDI, so a synth script that listens to port 1 plays it right away when nothing else is plugged in. click a key to play it: near the front is loud, further back is soft. drag across keys to play them in turn. right-click (or switch on LATCH) to hold notes, for drones and chords. with the keyboard window in front, the computer keyboard plays it like a piano: `A` is C, the row `A`–`'` the white keys, `W` `E` `T` `Y` `U` `O` `P` the black ones (by position, on any keyboard layout; the keys show their letters). `Z` / `X` move the octave, `C` / `V` the velocity.



- **no arc?** the ARC button (or F8) opens a virtual arc with four rings; scripts see it as an arc (in SYSTEM → DEVICES it is *monome arc virtual*). drag round a knob to turn it (clockwise is up; a whole turn of the mouse round it is a quarter turn of the knob, so it can be set finely), or scroll over it; hold ⇧ for fine turns, ⌘ for fast ones. right-click a knob to push it.



- **crow**: plug it in; it appears like on the device.
- **gamepads / joysticks**: USB and Bluetooth controllers work with scripts that support them.
- **keyboard**: the Mac keyboard is the norns keyboard (see controls).

---

11. the AUv3 plugin

Verdandi installs an Audio Unit called **Verdandi** (a music effect: audio in, audio out, MIDI in). it puts norns into a DAW track: the track's audio goes into norns, and norns' output comes back into the track.

1. start **Verdandi** (the plugin talks to it; norns keeps running in its own window).
2. in your DAW, insert **Verdandi** on a track — in Logic as an Audio FX or MIDI-controlled effect, in Ableton Live / Bitwig / Reaper from the Audio Unit plugins.
3. press play. the plugin window shows *connected*. while the DAW plays through the plugin, norns hears the track instead of your audio interface and plays into the track instead of your speakers. when the DAW stops, norns goes back to the interface.
4. MIDI sent to the plugin track reaches norns as the MIDI device **Verdandi AUv3** (choose it in a script's MIDI settings or in SYSTEM → DEVICES).

latency: about 8 ms, reported to the DAW so it compensates. norns follows the DAW's clock and

sample rate (it resamples internally). use one Verdandi plugin at a time. if Verdandi is not running, the plugin passes audio through unchanged.

if the plugin does not show up: start Verdandi once from Applications, then rescan plugins in the DAW (Logic: Plug-in Manager → Reset & Rescan).

---

12. files and logs

getting samples and scripts in: drop files or folders on any norns window (or choose File → Import). a note on the device says what happened.

a sample for a script: when a script asks for a file (say *buffer 1 sample* in PARAMS), the Finder's open panel comes up next to norns' own file browser: choose the sample anywhere on your Mac, and the script gets it (linked into `dust/audio/mac`, see below). or drop it on the norns window while the browser is up. *Cancel* leaves norns' browser to choose from dust as on the device.

- **audio** (wav, aiff, flac, ogg, mp3, and folders with audio in them) is *linked*: the files stay where they are, and `dust/audio/mac` gets a link to them, where sampler scripts find them like any other folder. nothing is copied, so there is nothing to clean up: deleting a link in `dust/audio/mac` never deletes your samples, and links whose files were moved or deleted are removed the next time you add something. if you move the originals or unplug their disk, norns cannot play them until they are back.
- hold ⌥ (Option) while dropping to *copy* the audio into `dust/audio` instead: for samples a saved preset should always find.
- **scripts** (a folder with a `.lua` file in it, or a single `.lua` file) are always copied to `dust/code`, because they belong to norns: maiden updates them, and norns starts the last one again on the next launch.
- **from other apps** (Voice Memos, Music, DAWs, sample browsers): many hand over a file only when it is dropped, or the sound itself. that is copied into `dust/audio`, as there is no original to link to; so is a file from a temporary folder, which the app would delete. if an app drags nothing norns can use, the note on the device says so: export the sound as a file first, or play it in with SYS or Audio Input.
- nothing is overwritten: a name that is already there gets a number.

getting things out: TAPE recordings are in `dust/audio/tape` (File → Show Tape Recordings); screenshots and GIFs of the screen go to your desktop.

file	what
`~/norns/norns.log`	matron and crone (Lua errors, devices)
`~/norns/sclang.log`	SuperCollider (engines, UGens)
`~/norns/jackd.log`	audio server
`~/norns/maiden.log`	maiden
`~/norns/auv3_bridge.log`	the AUv3 connection

to reset norns' settings, choose SYSTEM → RESET (or quit Verdandi and delete `~/norns/dust/data/system.*`). your scripts and audio are untouched.

13. troubleshooting

problem	what to do
a script says "version … required"	it needs a newer norns than this Verdandi contains: a new version of Verdandi brings it. (Verdandi before 0.1.1 always said 000000 here.)
"Verdandi cannot be opened"	*System Settings* → *Privacy & Security* → *Open Anyway* (see installing).
SUPERCOLLIDER FAIL on the first start	macOS was still checking the bundled libraries; quit and start Verdandi again.
no sound	check the default output in System Settings → Sound, the levels in the MIX menu, and `jackd.log`.
no input	allow the microphone for Verdandi (Privacy & Security → Microphone); check the input level in MIX.
a second Verdandi does not start	only one at a time; close the other window first.
a script misbehaves	open maiden's REPL or `norns.log` for the error.
maiden does not load	use `http://127.0.0.1:5000/maiden` (not `localhost`), with Verdandi running.

14. differences from the device

- SYSTEM shows DEVICES, AUDIO, MODS, RESET and RESTART. Wi-Fi and the password are the Mac's job, scripts are updated in maiden, and Verdandi itself with a new version of the app. the norns release it contains (scripts that ask for a newer norns check it) is written to `~/norns/version.txt` at every start; this one is 260819.
- the encoders are endless: E1 goes round the menu pages (after PARAMS comes MIX again). inside a page and in scripts, values stop where the page or script lets them stop.
- SYSTEM → RESTART restarts norns and SuperCollider; SLEEP quits Verdandi (it does not put the Mac to sleep).
- no battery or screen settings: there is no battery, and the screen is a window.
- audio device and sample rate come from macOS (see audio).
- maiden and the consoles are local-only (127.0.0.1); OSC (port 10111) is reachable on your network, so phone controllers like TouchOSC work as on the device.

15. uninstalling

delete `/Applications/Verdandi.app`. your norns folder `~/norns` (scripts, audio, data) stays; delete it too if you no longer need it.

16. source and licenses

Verdandi is free software. the app contains, each under its own license: norns (GPL-3.0, monome and contributors), maiden (GPL-3.0), SuperCollider (GPL-3.0), sc3-plugins (GPL-2.0 or later), JACK (GPL-2.0 / LGPL-2.1), Lua (MIT), lua-cjson (MIT), liblo (LGPL-2.1), libmonome (ISC), cairo (LGPL-2.1 / MPL-1.1), SDL (zlib), libsndfile (LGPL-2.1), libsamplerate (BSD-2), nng (MIT), Boost (BSL-1.0), yaml-cpp (MIT), GNU coreutils, findutils, sed and grep (GPL-3.0), and their dependencies. the folder **Licenses** (in the dmg next to the app, and in `Verdandi.app/Contents/Resources/Licenses`) has the license texts of every part and `PACKAGES.txt`, the list of all of them with version, license and where the exact source is. the complete source of the port, with build instructions (`readme-macos.md`, `macos/setup-deps.sh`, `macos/make-app.sh`), is published with each release; GPL-3.0 grants you the right to get, study, change and share it.

Verdandi runs norns, made by monome and its community. norns and monome are names of monome; Verdandi is an independent community project, not endorsed by or affiliated with monome.

appendix: script builder reference

blocks

every block of the toolbox, by group. ( ) is a slot for a value, […] a choice or a field. Logic, loops, math, text, lists, variables and functions are Blockly's own.

Events

block	what it does
when the script starts	runs once when the script loads: set things up here
every [1/4 beat]	repeats in time with the norns clock (tempo in PARAMETERS > CLOCK)
when key [K2] is [pressed]	runs when a key of the norns is pressed or released
when encoder [E2] turns	runs on every step of an encoder; "turn amount" is +1 or -1 (more when turned fast)
turn amount	how far the encoder turned: positive clockwise, negative counter-clockwise
draw the screen	draws the 128 x 64 screen, 15 times a second; it is cleared before each drawing

Sound

block	what it does
play note ( ) velocity ( ) for ( ) beats	a MIDI note number (60 is middle C), velocity 0 to 127; nothing plays when the note is empty (a rest)
play frequency ( ) Hz velocity ( ) for ( ) beats	
start note ( ) velocity ( )	the note sounds until "stop note" with the same number: for keys held down
stop note ( )	
stop all notes	
set sound [...] to ( )	any knob of the sound page; it is also in PARAMETERS > sound
change sound [...] by ( )	adds to a knob, kept inside its range: "change sound filter1 cutoff by turn amount x 50"
sound [...]	
play MIDI notes with the sound	notes from the MIDI device in slot 1 play the sound, with their velocity
load sound file [my_sound.wav] into sample slot ( )	a file in dust/audio (or a full path) for the sample module of the sound page
voices sounding	how many notes sound right now (held or ringing out)

Music

block	what it does
note ( ) of scale [Major] from ( )	step 1 is the root note; higher steps climb the scale through the octaves
name of note ( )	for example 60 gives C4
sequence of ( )	a sequins: store it in a variable, then take "next from" it
next from [melody]	the next value of a sequence; after the last one it starts over

Sequencing

block	what it does
count [step] from 1 to （ ）, then again	adds 1 each time, back to 1 after the last: the step of a sequence
step （ ） of rhythm [x..x..x.x...x...] is a hit	x (or any letter) is a hit, a dot a rest; after the last step it starts over
step （ ） of [60 . 63 67 . 70 67 63]	numbers with spaces between them; a dot is a rest: "play note" plays nothing then
step （ ） of （ ） hits in （ ） steps, shifted by （ ）, is a hit	a euclidean rhythm: the hits as evenly spread as they can be (3 in 8 is the tresillo)
chance of （ ） percent	true that often, at random: 50 is a coin toss
every （ ）. time this runs	true the first time, then every n-th time
notes of the [Major] chord on （ ）	a list of note numbers, the root first
note （ ） of （ ） going [up]	an arpeggio: step 1, 2, 3 ... walks through the notes
（ ） snapped to [Major] on （ ）	the nearest note of the scale: any number becomes a note that fits

Time

block	what it does
wait （ ） beats	only inside "every ..." blocks
set tempo to （ ） BPM	
tempo (BPM)	

Looper

block	what it does
set up looper [1] for loops up to [30] seconds, recording [the audio input]	a looper on a softcut voice; set it up in "when the script starts"
looper [1]: [record a new loop]	record, then finish: the time in between is the loop. overdub records on top while it plays
looper [1] [speed (1 normal, -1 backwards)] ( )	speed 2 is an octave up, 0.5 an octave down, -1 backwards. a length in beats makes every recording that long
looper [1] jump to ( ) (0 start, 1 end)	
load sound file [my_loop.wav] into looper [1]	a file in dust/audio (or a full path); it plays at once, as long as the looper allows
position of looper [1] (0 to 1)	
length of looper [1] (seconds)	
looper [1] is [recording]	
waveform of looper [1] at x ( ) y ( ) width ( ) height ( )	the loop as a picture, with the playhead

MIDI

block	what it does
when a MIDI note [starts] arrives	notes from the MIDI device in slot 1 (SYSTEM > DEVICES > MIDI)
MIDI note number	
MIDI velocity	
send MIDI note ( ) velocity ( ) for ( ) beats on channel ( )	to the MIDI device in slot 1; the note ends by itself
when MIDI CC [1] changes	a knob of a MIDI controller (device slot 1); "CC value" is 0 to 127
CC value	
send MIDI CC ( ) value ( ) on channel ( )	

Grid

block	what it does
when a grid key is [pressed]	"grid x" (1 to 16, left to right) and "grid y" (1 to 8, top down) say which
grid x	
grid y	
grid key x（ ） y（ ） is held	
draw the grid	30 times a second, all lights off first: light what should be lit
light grid x（ ） y（ ） brightness（ ）	brightness 0 (off) to 15
when arc ring [1] turns	"arc turn amount" says how far, positive clockwise
arc turn amount	
draw the arc	
light arc ring [1] light（ ） brightness（ ）	64 lights per ring, 1 at the top
light arc ring [1] from（ ） to（ ） (0 to 1) brightness（ ）	a stretch of the ring: from 0 to the value of a control draws a meter

Screen

block	what it does
write ( ) at x ( ) y ( ) brightness ( ) [from the left]	x 0 to 127, y 0 to 63 (y is the bottom of the letters), brightness 0 to 15
[filled] rectangle at x ( ) y ( ) width ( ) height ( ) brightness ( )	
[filled] circle at x ( ) y ( ) radius ( ) brightness ( )	
line from x ( ) y ( ) to x ( ) y ( ) brightness ( )	
pixel at x ( ) y ( ) brightness ( )	
text size ( )	for the "write" blocks after it; 8 is the norns size
frame number	counts up with every drawing (15 a second): use it to move things
beats since start	the clock position in beats: animations that move with the tempo
show page ( ) of the screen design	elements on page 0 always show; the others only on their page
screen design page	
set screen element [...] [...] to ( )	the script decides this value now (it stops moving by itself)
let screen element [...] [...] move by itself again	
[show] screen element [...]	
set off the note animations	what moves "on each note" moves now, as if a note played

Widgets

block	what it does
dial for control [cutoff] at x ( ) y ( ) size ( )	the norns dial, with the control's name and value
[upright] slider for control [cutoff] at x ( ) y ( ) width ( ) height ( )	
play symbol at x ( ) y ( ) size ( ) showing ( )	a play triangle when the value is true, a stop square when it is false
page dots: page ( ) of ( )	the dots on the right that show which page is on (keep the page in a variable, change it with E1)
tabs [play, sound, loop] showing tab ( )	tab names with commas between them, across the top
list [one, two, three, four] at x ( ) y ( ) with item ( ) picked	a scrolling list, the picked item bright

Controls

block	what it does
add control [cutoff] from [0] to [100] starting at [50]	a parameter in the PARAMETERS menu; it can be mapped to MIDI and saved in PSETs. add controls in "when the script starts"
control [cutoff]	
set control [cutoff] to （ ）	
change control [cutoff] by （ ）	adds to a control, kept inside its range: "change control cutoff by turn amount"
add choice [mode] with [up, down, random] starting at [1]	a control that picks one of these (commas between them); "control mode" is its number, 1 for the first
add switch [hold] starting [off]	on (1) or off (0); MIDI mappable
shown value of control [mode]	what PARAMETERS shows: the name of a choice, a value with its unit
when control [cutoff] changes	from an encoder, MIDI, an LFO or a block: "new value" is what it is now
new value	
LFO moves control [cutoff] from （ ） to （ ） every （ ） beats, [smoothly]	any control, sound knobs too (control "filter1 cutoff"); in time with the clock

Data

block	what it does
remember （ ） as [pattern]	kept in the script's data folder: still there the next time the script runs
remembered [pattern], else （ ）	

sound modules and effects

every module of the sound page; its knobs are its params. option params are choices.

always there

module	what it does	in / out	knobs
voice	what each played note gives the patch: its pitch, its gate (held or not), its velocity and a random number of its own	– / pitch (Hz), gate, velocity, random	voices, mode (poly, mono, legato), glide, bend
output	where the sound of a voice leaves the patch, into the effects	in, pan / –	level, pan, spread

sources

module	what it does	in / out	knobs
oscillator	sine, triangle, saw or square; fm bends its pitch by the fm input	pitch, fm, pwm / out	wave (sine, triangle, saw, square), tune, fine, width, fm depth, level
supersaw	seven detuned saws: big leads and pads	pitch / out	tune, detune, level
fm operator	a sine at a ratio of the pitch; another operator into pm makes bells and keys	pitch, pm / out	ratio, fine, pm depth, level
noise	white, pink or brown noise	– / out	color (white, pink, brown), level
string	a plucked string (Karplus-Strong), struck by the gate	pitch, strike / out	decay, damp, level
vowel	a voice-like tone that sings a, e, i, o, u (in between too)	pitch, vowel / out	vowel, level
sample	plays a sound file loaded into a slot (the "load sample" block), at the pitch of the note	pitch, start / out	slot, start, end, loop (off, on), follow pitch, level

shapers

module	what it does	in / out	knobs
filter	lowpass, highpass, bandpass or notch; mod moves the cutoff in octaves	in, mod, key / out	type (lowpass, highpass, bandpass, notch), cutoff, resonance, mod depth, key follow
amp	the in signal, as loud as the cv input says (an envelope, usually)	in, cv / out	level
mixer	four inputs into one	in 1, in 2, in 3, in 4 / out	level 1, level 2, level 3, level 4
drive	soft saturation, wavefolding or hard clipping	in, mod / out	type (saturate, fold, clip), amount, level
bitcrush	fewer bits and a lower sample rate: lo-fi	in / out	bits, rate
resonator	a tuned comb: rings the in signal at the note's pitch (try noise in)	in, pitch / out	tune, decay, level
ring mod	a times b	a, b / out	
slew	smooths changes: steps become glides	in / out	time
scale + offset	in times amount, plus offset: turn a modulation up, down or upside down	in / out	amount, offset

modulators

module	what it does	in / out	knobs
envelope	attack, decay, sustain, release while the gate is held; 0 to 1	gate / out	attack, decay, sustain, release, curve, velocity
lfo	a slow wave, -1 to 1, started with each note	rate mod / out	wave (sine, triangle, saw, square, random), rate, depth
sample + hold	holds a new random value (or the in signal) at each tick: random steps	in / out	rate
drift	wanders slowly and smoothly, -1 to 1: life for pads and tuning	– / out	rate, depth

effects

effect	what it does	knobs
main	the line input into the effects, and the level of everything	line in, level
filter	a filter over everything	type (lowpass, highpass), cutoff, resonance
drive	saturation over everything	amount, mix
bitcrush	lo-fi over everything	bits, rate, mix
tape	wow, flutter and tape saturation	wow, flutter, saturation
chorus	a slowly moving copy next to the sound	rate, depth, mix
phaser	sweeping notches	rate, depth, mix
pitch shift	a copy shifted up or down (octaves, fifths)	shift, mix
grains	scatters the sound into a cloud of small grains	density, size, mix
delay	echoes; spread puts the right side later	time, feedback, tone, spread, mix
reverb	a room to a hall	size, damp, mix
compressor	evens out the level: punch, or glue	threshold, ratio, makeup
width	narrower (0) or wider (2) stereo	width

screen elements

element	what it draws	properties
text (shapes)	words; {tempo}, {note}, {beat}, {step}, {page} and {control name} show live values	text, x, y, size, brightness, align (left, centre, right), page
rectangle (shapes)	a box, filled or outlined	x, y, width, height, brightness, drawn (filled, outlined), page
circle (shapes)	a circle, filled or outlined	x, y, radius, brightness, drawn (filled, outlined), page
line (shapes)	a line from one point to another	from x, from y, to x, to y, thickness, brightness, page
arc (shapes)	part of a ring: from 0 (the top) round to 1	x, y, radius, from, to, thickness, brightness, page
meter (values)	a bar filled as far as its value (0 to 1): give it a control, the sound level or the beat	x, y, width, height, value, brightness, direction (across, up), page
dial (values)	the norns dial of a control, with its name and value	control, x, y, size, page
slider (values)	the norns slider of a control	control, x, y, width, height, direction (up, across), page
looper wave (values)	the loop of a looper (set up in the blocks), with its playhead	looper, x, y, width, height, page
steps (values)	a row of steps, the current one bright: the counter of a sequence (a variable of the blocks)	step variable, steps, x, y, width, height, brightness, page
rings on notes (moving)	every note sends a ring out from the centre, fading as it grows	x, y, reach, lasts, brightness, page
sparks on notes (moving)	every note throws sparks that fall and fade	x, y, spread, sparks, gravity, brightness, page
orbit (moving)	dots going round, once every so many beats	x, y, radius, dots, round in, dot size, brightness, page

wave (moving)	a sine line across the screen, flowing with the beat	y, height, waves, flows in, brightness, page
spiral (moving)	a turning spiral	x, y, size, turns, turns in, brightness, page
lissajous (moving)	the figure two waves draw together, slowly turning	x, y, width, height, across, up, turns in, brightness, page
stars (moving)	flying through stars	stars, speed, brightness, page
rain (moving)	falling drops	drops, speed, length, brightness, page
snow (moving)	flickering pixels, like an old screen	x, y, width, height, density, brightness, page
bouncing ball (moving)	a ball bouncing off the edges	size, speed, brightness, page

how a number moves

moves	how	settings
with the beat	goes from "from" to "to" and back, in time with the clock	from, to, every, shape (sine, triangle, saw, square, random)
over time	the same, in seconds, whatever the tempo	from, to, every, shape (sine, triangle, saw, square, random)
on each note	jumps to "to" when a note plays, then glides back to "from"	from, to, back in
with the sound level	"from" when silent, "to" when loud	from, to
with a control	"from" at the control's lowest, "to" at its highest	from, to, control
with a looper's playhead	"from" at the start of the loop, "to" at its end	from, to, looper